컴파일러의 기초 project 4 보고서

전기정보공학부 2021-15905 홍성준

December 19, 2025

1 Design

Global variables, arithmetic operations, control flows, function declaration, struct declaration, array operation 등을 허용하며, yacc과 lex를 사용해 IR code generator를 구성하였다. input stream으로 .c source code 문자열을 lexer를 거쳐 토큰이 만들어지고, 각 토큰이 parser에게 전달되어 LR parsing을 진행함과 동시에 yacc의 midrule action으로 intermediate code를 생성한다.

2 Basic Features

기본적으로, unary → ID production에 의한 reduce에서 ID에 대해 symbol table lookup을 진행한 후 offset을 찾는다. 만약 global scope에 정의되어 있었으면 Lglob+offset을 stack top에 만들고, function scope에 정의되어 있었으면 fp+offset을 stack top에 만들도록 하였다. 각 연산에 따라 실제 값이 필요한 경우에는 fetch를 추가하여 주소가 아닌 값과 연산을 진행할 수 있도록 하였다.

2.1 Function Call

```
| unary '(' {
  func_prologue($1);
  $<intval>$ = label_offset;
  if ($1->funcid != write_int_id && $1->funcid != write_char_id&&
    $1->funcid != write_string_id) label_offset++;
} args ')' {
  $$ = access_function($1, $4);
  func_call($1);
  if ($1->funcid != write_int_id && $1->funcid != write_char_id&&
    $1->funcid != write_string_id)
  make_label_offset($<intval>3);
  $$->deref = 1;
}
| unary '(' {
  func_prologue($1);
  $<intval>$ = label_offset;
  if ($1->funcid != write_int_id && $1->funcid != write_char_id&&
    $1->funcid != write_string_id) label_offset++;
} ')' {
  $$ = access_function($1, NULL);
  func_call($1);
  if ($1->funcid != write_int_id && $1->funcid != write_char_id&&
    $1->funcid != write_string_id)
  make_label_offset($<intval>3);
  $$->deref = 1;
}
```

Figure 1: Midrule action for function call

function call을 수행하는 midrule action은 Figure 1과 같다. 먼저 argument에 대한 code를 생성하기 전, `func_prologue`에서 리턴값을 넣을 공간을 `shift_sp(1)`를 통해 만든 후, 호출이 끝난 후 돌아올 label과 현재 fp를 스택에 저장한다. argument에 대한 code 생성이 끝나면, `func_call`에서 argument 사이즈만큼 sp에서 빼 fp 값을 수정하고 jump를 통해 호출할 함수의 label로 control flow를 전환시킨다.

2.2 Assignment

```
expr
  : unary '=' {
    if ($1->type->typeclass == TYPE_STRUCT){
      str_assign_prologue($1->type);
    }
    else {
      push_reg("sp");
      fetch(NULL, 0);
    }
  } expr {
    if ($4 == int_tdecl) $$ = int_tdecl_const;
    else if ($4 == char_tdecl) $$ = char_tdecl_const;
    else $$ = $4;

    if ($1->type->typeclass == TYPE_STRUCT){
      if (!func_flag) str_assign($4);
    }
    else assign();
    fetch(NULL, 0);
    func_flag = 0;
  }
```

Figure 2: Midrule action for assignment

assignment를 위한 midrule action은 위 Figure 2와 같이 구현하였다. 여러 =에 대해 assign을 수행하기 위해 stack top 값을 복사하여 assign을 진행한 후 fetch를 통해 할당된 값을 stack top에 유지할 수 있도록 하였다.

이외에도, if/else branch와 arithmetic operation, built-in function 등 기본적인 연산부터 복잡하게 엮인 control flow까지 모두 동작을 무사히 수행할 수 있도록 코드를 생성하는데 성공하였다.

3 Advanced Features

3.1 Loops

Non-nested while loops, Non-nested for loops, Nested while/for loops, break/continue 기능을 모두 구현하였다. loop control flow를 위한 midrule action을 Figure 3과 같이 구현하였다. nested loop을 처리하기 위해 midrule action의 intval에 local_offset을 할당해준 후, stmt가 끝난 후 action에서 해당 값으로 label을 만들 수 있도록 하였다. 이를 통해 stmt안에서 만들어진 loop과 label이 겹치는 것을 피할 수 있었다. for loop의 경우 shift 순서를 따르기 위해 한 loop당 총 4개의 label을 사용하도록 하였다. for/while loop의 control flow는 다음 Listing 1에 나타내었다. 또한, break 및 continue를 만났을 때 jump할 label을 nested loop에서도 저장하기 위해서 break, continue stack을 구현하였다. loop이 끝나는 시점에 pop을 진행하고, 들어갈 때 push를 수행해 loop 밖과 loop의 시작 지점 label을 저장하도록

하였다. 최종적으로 break나 continue를 만나면 각각의 stack top을 참조해 jump 코드를 생성하도록 하였다.

```
L0:
    test_expr
    branch_false L3
    jump L2
L1:
    update_expr
    jump L0
L2:
    body_statements
    jump L1
L3:
```

```
L0:
    test_expr
    branch_false L1
    body_statements
    jump L0
L1:
```

Listing 1: Control flow of `for` and `while`

```
| WHILE {
  $<intval>$ = label_offset;
  push_labels(label_offset, label_offset+1);
  make_label();
} '(' expr {
  branch(FALSE, NULL, 0, label_offset++);
} ')' stmt {
  jump(NULL, 0, 0, $<intval>2);
  make_label_offset($<intval>2+1);
  pop_cont();
  pop_break();
}
| FOR '(' expr_e {
  $<intval>$ = label_offset;
  push_labels(label_offset+1, label_offset+3);
  make_label();
} ';' expr_e {
  branch(FALSE, NULL, 0, $<intval>4 + 3);
  jump(NULL, 0, 0, $<intval>4 + 2);
  make_label();
} ';' expr_e ')' {
  jump(NULL, 0, 0, $<intval>4);
  make_label();
  label_offset++;
} stmt {
  jump(NULL, 0, 0, $<intval>4+1);
  make_label_offset($<intval>4+3);
  pop_cont();
  pop_break();
}
| BREAK ';' { jump(NULL, 0, 0, top_break()); }
| CONTINUE ';' { jump(NULL, 0, 0, top_cont()); }
```

Figure 3: Midrule action for loops

3.2 Struct Operations

Assignment operation between struct types, Functions with struct return types, Functions accepting struct as parameters 세 가지 기능을 아래 Figure 4와 같이 모두 구현하였다. struct size를 n이라 했을 때, struct field 안에 array나 다른 struct 변수가 있는 상황을 고려해 offset을 0부터 n-1까지 증가시키며 assign 코드를 반복적으로 생성하도록 하였다.

Assignment의 경우, base1+0, ... base1+n-1을 스택에 모두 쌓은 뒤 할당할 struct 변수의 모든 field에 대해, base2+n-1부터 base2+0까지 하나씩 push 및 assign 코드를 생성하는 과정을 반복적으로 수행하였다. struct return type의 경우에는, return value 바로 아래부분에 base+0, ..., base+n-1이 존재하므로, 해당 값을 stack top에 fetch한 뒤 assign하는 코드를 반복적으로 생성해 총 n개의 assign을 진행할 수 있도록 하였다. struct parameter의 경우에는, `str_passarg`와 같이 size가 n인 struct를 1 size를 갖는 n개의 argument를 전달하듯이 stack top에 차곡이 쌓을 수 있도록 하였다. callee에서는 일반적인 parameter 사용과 동일하게, offset을 통해 해당 field 값을 참조할 수 있도록 하였다.

```c
void str_assign_prologue(decl_t *strdecl) {
  for (int i = 0; i < strdecl->size; i++) {
    push_reg("sp");
    fetch(NULL, 0);
    if (i > 0) {
      push_const_int(1);
      gen_add();
    }
  }
}

void str_assign(decl_t *strdecl) {
  for (int i = 0; i < strdecl->size; i++) {
    push_reg("sp");
    push_const_int(-1 - i);
    gen_add();
    fetch(NULL, 0);
    push_reg("sp");
    push_const_int(-1);
    gen_add();
    fetch(NULL, 0);
    if (i < strdecl->size - 1) {
      push_const_int(strdecl->size - 1 - i);
      gen_add();
    }
    fetch(NULL, 0);
    assign();
  }
}
```

```c
void str_ret(id *idptr) {
  decl_t *strdecl = lookup_funcscope(idptr);
  if (strdecl) {
    // local variable
    for (int i = 0; i < strdecl->size; i++) {
      push_reg("fp");
      push_const_int(-3 - i);
      gen_add();
      fetch(NULL, 0);
      push_reg("fp");
      push_const_int(1 + strdecl->offset + strdecl->size - 1 - i);
      gen_add();
      fetch(NULL, 0);
      assign();
    }
  } else {
    // global variable
    strdecl = find_decl(scope[SCOPE_GLOB], idptr);
    for (int i = 0; i < strdecl->size; i++) {
      push_reg("fp");
      push_const_int(-3 - i);
      gen_add();
      fetch(NULL, 0);
      push_const_label_offset("Lglob", strdecl->offset);
      push_const_int(strdecl->size - 1 - i);
      gen_add();
      fetch(NULL, 0);
      assign();
    }
  }
}

void str_passarg(id *idptr) {
  decl_t *strdecl = lookup(idptr)->type;
  for (int i = 0; i < strdecl->size; i++) {
    load_var(idptr);
    if (i > 0) {
      push_const_int(i);
      gen_add();
    }
    fetch(NULL, 0);
  }
}
```

Figure 4: Implementation of struct operations